

CS-114 Fundamentals of Programming

Character Arrays - Strings - Structures

Instructor: Jahan Zeb

Department of Computer & Software Engineering (DC&SE)

College of E&ME

NUST

Character Arrays

□ Strings

□ Arrays of characters

- All strings end with **null** (' \0 ')

- Examples

- **char string1[] = "hello";**

- **Null** character implicitly added

- **string1** has 6 elements

- **char string1[] = { 'h', 'e', 'l', 'l', 'o', '\0' };**

- Subscripting is the same

- String1[0]** is 'h'

- string1[2]** is 'l'

Examples Using Arrays

□ Input from keyboard

```
char string2[ 10 ];  
cin >> string2;
```

- Puts user input in string
 - Stops at first whitespace character
 - Adds **null** character

□ Printing strings

- **cout << string2 << endl;**
 - Does not work for other array types
- Characters printed until **null** found

```

1 // Fig. 4_12: fig04_12.cpp
2 // Treating character arrays as strings.
3 #include <iostream>
4
5 using std::cout;
6 using std::cin;
7 using std::endl;
8
9 int main()
10 {
11     char string1[ 20 ], //
12     char string2[] = "string literal": //
13
14     // read string from user into array
15     cout << "Enter the string \"hello\" and press Enter\n";
16     cin >> string1; // reads "hello" [space terminates input]
17
18     // output strings
19     cout << "string1 is: " << string1
20         << "\nstring2 is: " << string2;
21
22     cout << "\nstring1 with spaces between characters is:\n";
23

```

Two different ways to declare strings. **string2** is initialized, and its size determined automatically.

Examples of reading strings from the keyboard and printing them out.

```

24 // output characters until null character is reached
25 for ( int i = 0; string1[ i ] != '\0'; i++ )
26     cout << string1[ i ] << ' ';
27
28 cin >> string1; // reads "there"
29 cout << "\nstring1 is: " << string1 << endl;
30
31 return 0; // indicates successful termination
32
33 } // end main

```

Can access the characters in a string using array notation. The loop ends when the **null** character is found.

```

Enter the string "hello there": hello there
string1 is: hello
string2 is: string literal
string1 with spaces between characters is:
h e l l o
string1 is: there

```

Fundamentals of Characters and Strings

□ Character constant

- Integer value represented as character in single quotes
- `'z'` is integer value of **z**
 - **122** in ASCII

□ String

- Series of characters treated as single unit
- Can include letters, digits, special characters `+`, `-`, `*` ...
- String literal (string constants)
 - Enclosed in double quotes, for example:
`"I like C++"`
- Array of characters, ends with null character `'\0'`
- String is constant pointer
 - Pointer to string's first character
 - Like arrays

Fundamentals of Characters and Strings

□ String assignment

- Character array

- `char color[] = "blue";`

- Creates 5 element `char` array `color`

- last element is `'\0'`

- Variable of type `char *`

- `char *colorPtr = "blue";`

- Creates pointer `colorPtr` to letter `b` in string `"blue"`

- `"blue"` somewhere in memory

- Alternative for character array

- `char color[] = { 'b', 'l', 'u', 'e', '\0' };`

Fundamentals of Characters and Strings

□ Reading strings

- Assign input to character array **word[20]**

```
cin >> word
```

- Reads characters until whitespace or EOF
- String could exceed array size

```
cin >> setw( 20 ) >> word;
```

- Reads 19 characters (space reserved for ' \0 ')

Fundamentals of Characters and Strings

□ `cin.getline`

- Read line of text
- `cin.getline( array, size, delimiter );`
- Copies input into specified **array** until either
 - One less than **size** is reached
 - **delimiter** character is input
- Example

```
char sentence[ 80 ];  
cin.getline( sentence, 80, '\n' );
```

String Manipulation Functions of the String-handling Library

- String handling library **<cstring>** provides functions to
 - Manipulate string data
 - Compare strings
 - Search strings for characters and other strings
 - Tokenize strings (separate strings into logical pieces)

String Manipulation Functions of the String-handling Library

<code>char *strcpy(char *s1, const char *s2);</code>	Copies the string s2 into the character array s1 . The value of s1 is returned.
<code>char *strncpy(char *s1, const char *s2, size_t n);</code>	Copies at most n characters of the string s2 into the character array s1 . The value of s1 is returned.
<code>char *strcat(char *s1, const char *s2);</code>	Appends the string s2 to the string s1 . The first character of s2 overwrites the terminating null character of s1 . The value of s1 is returned.
<code>char *strncat(char *s1, const char *s2, size_t n);</code>	Appends at most n characters of string s2 to string s1 . The first character of s2 overwrites the terminating null character of s1 . The value of s1 is returned.
<code>int strcmp(const char *s1, const char *s2);</code>	Compares the string s1 with the string s2 . The function returns a value of zero, less than zero or greater than zero if s1 is equal to, less than or greater than s2 , respectively.

String Manipulation Functions of the String-handling Library

□ Copying strings

– **char *strcpy(char *s1, const char *s2)**

- Copies second argument into first argument
 - First argument must be large enough to store string and terminating null character

– **char *strncpy(char *s1, const char *s2, size_t n)**

- Specifies number of characters to be copied from string into array
- Does not necessarily copy terminating null character

```

1
2 // Using strcpy and strncpy.
3 #include <iostream>
4
5 using std::cout;
6 using std::endl;
7
8 #include <cstring> // prototypes for strcpy and strncpy
9
10 int main()
11 {
12     char x[] = "Happy Birthday to You";
13     char y[ 25 ];
14     char z[ 15 ];
15
16     strcpy( y, x ); // copy contents of x into y
17
18     cout << "The string in array x is: " << x
19         << "\nThe string in array y is: "
20
21     // copy first 14 characters of x
22     strncpy( z, x, 14 ); // does not
23     z[ 14 ] = '\0'; // append '\0' to
24
25     cout << "The string in array z is: " << z << endl;

```

`<cstring>` contains prototypes for **strcpy** and **strncpy**.

Copy entire string in array **x** into array **y**.

Copy first 14 characters of
Append terminating null
character.

that
ting

```
26
27     return 0; // indicates successful termination
28
29 } // end main
```

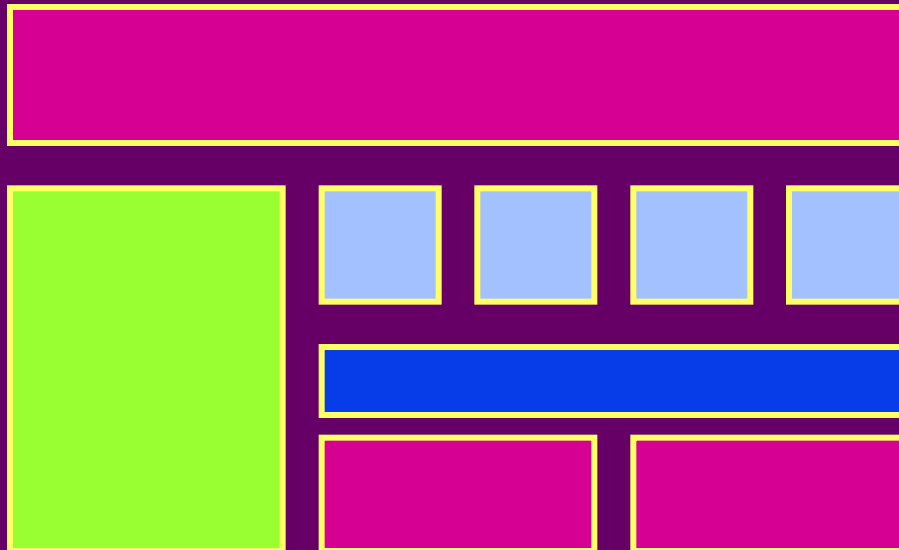
The string in array x is: Happy Birthday to You
The string in array y is: Happy Birthday to You
The string in array z is: Happy Birthday

String to copy.

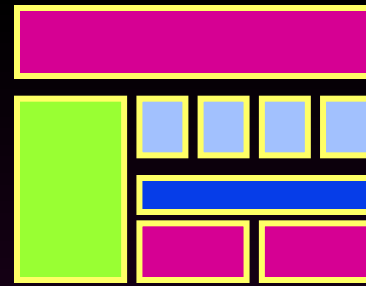
Copied string using **strcpy**.

Copied first 14 characters
using **strncpy**.

Structures

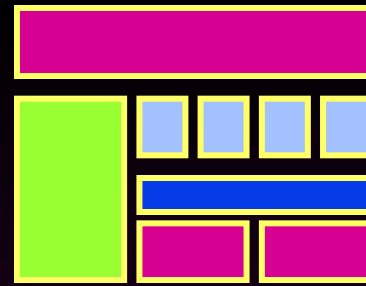


Structures



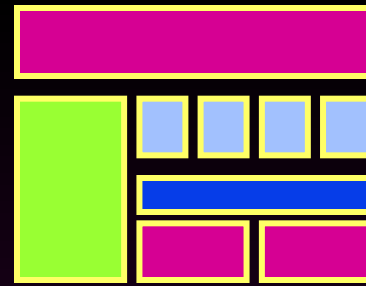
- A **Structure** is a collection of related data items, possibly of different types.
- A structure type in C++ is called **struct**.
- A **struct** is **heterogeneous** in that it can be composed of data of different types.
- In contrast, **array** is **homogeneous** since it can contain only data of the same type.

Structures



- Structures hold data that belong **together**.
- Examples:
 - Student record: student id, name, major, gender, start year, ...
 - Bank account: account number, name, currency, balance, ...
 - Address book: name, address, telephone number, ...
- In database applications, structures are called records.

Structures



- Individual components of a struct type are called **members** (or **fields**).
- Members can be of **different types** (simple, array or struct).
- A struct is named as a whole while individual members are named using field identifiers.
- Complex data structures can be formed by defining **arrays of structs**.

Structures

- ◆ Recall that elements of arrays must all be of the same type



A horizontal array of 10 cells. The first cell contains the text 'scores :'. The subsequent cells contain the values 85, 79, 92, 57, 68, 80, an ellipsis '...', and two empty cells. Below the cells are indices 0 through 99, with the ellipsis cell corresponding to index 98. Three red arrows originate from the word 'same' in the bullet point above and point to the cells containing 92, 57, and 68, illustrating that all elements in the array are of the same type.

scores :	85	79	92	57	68	80	...		
	0	1	2	3	4	5		98	99

- ◆ In some situations, we wish to group elements of different types



A horizontal structure with 7 cells. The first cell contains the text 'employee'. The subsequent cells contain the values 'R. Jones', '123 Elm', '6/12/55', '\$14.75', and one empty cell. Three red arrows originate from the word 'different' in the bullet point above and point to the cells containing 'R. Jones', '123 Elm', and '6/12/55', illustrating that the structure contains elements of different types.

employee	R. Jones	123 Elm	6/12/55	\$14.75	
----------	----------	---------	---------	---------	--

Structures

- ◆ RECORDS are used to group related components of different types
- ◆ Components of the record are called fields

employee	Name	123	6/12/95	PKR 40K
----------	------	-----	---------	---------

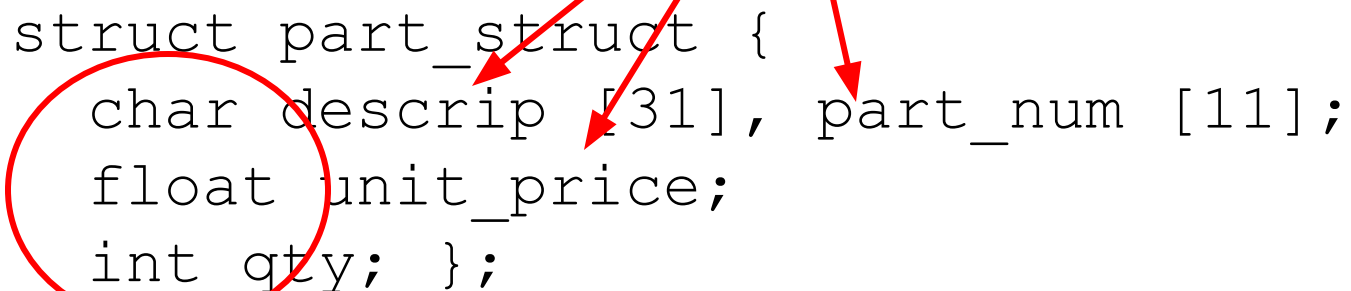
- ◆ In C++
 - record called a struct (structure)
 - fields called members

Structures

◆ C++ struct

- structured data type
- fixed number of components
- elements accessed by name, not by index
- components may be of different types

```
struct part_struct {  
    char descrip [31], part_num [11];  
    float unit_price;  
    int qty; };
```



Declaring struct Variables

◆ Given

```
struct part_struct {  
    char descrip [31], part_num [11];  
    float unit_price;  
    int qty; };
```

◆ Declare :

Use struct name as a type.

```
part_struct new_part, old_part;
```

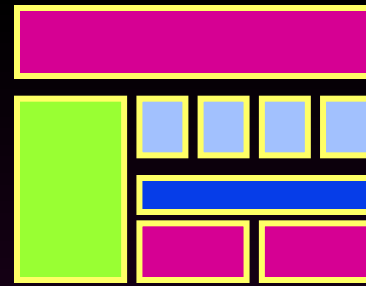
Accessing Components

- ◆ Use the name of the record
the name of the member
separated by a dot .

```
old_part.qty = 5;  
cout << new_part.descrip;
```

- ◆ The dot is called the member selector

struct basics



- Definition of a structure:

```
struct <struct-type>{  
    <type> <identifier_list>;  
    <type> <identifier_list>;  
    ...  
} ;
```

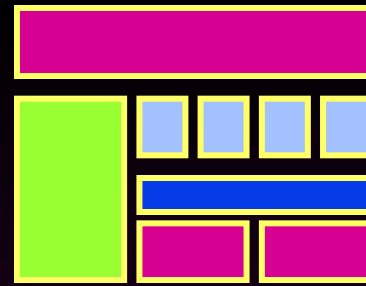
Each identifier
defines a member
of the structure.

- Example:

```
struct Date {  
    int day;  
    int month;  
    int year;  
} ;
```

The “Date” structure
has 3 members,
day, month & year.

struct examples



- Example:

```
struct StudentInfo{  
    int Id;  
    int age;  
    char Gender;  
    double CGA;  
};
```

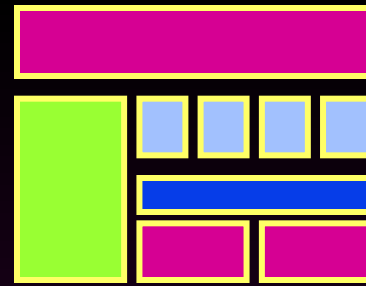
The “StudentInfo”
structure has 4 members
of different types.

- Example:

```
struct StudentGrade{  
    char Name[15];  
    char Course[9];  
    int Lab[5];  
    int Homework[3];  
    int Exam[2];  
};
```

The “StudentGrade”
structure has 5
members of
different array types.

struct examples



- Example:

```
struct BankAccount{  
    char Name[15];  
    int AccountNo[10];  
    double balance;  
    Date Birthday;  
};
```

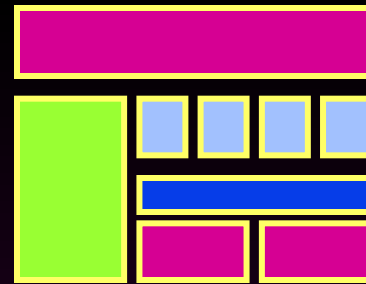
The “BankAccount”
structure has simple,
array and structure
types as members.

- Example:

```
struct StudentRecord{  
    char Name[15];  
    int Id;  
    char Dept[5];  
    char Gender;  
};
```

The “StudentRecord”
structure has 4
members.

struct basics

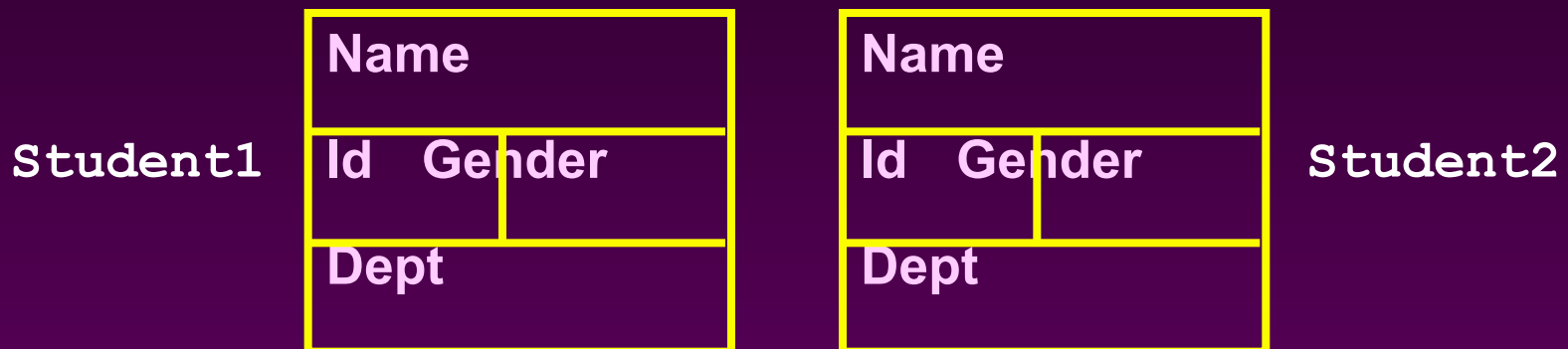


- Declaration of a variable of struct type:

```
<struct-type> <identifier_list>;
```

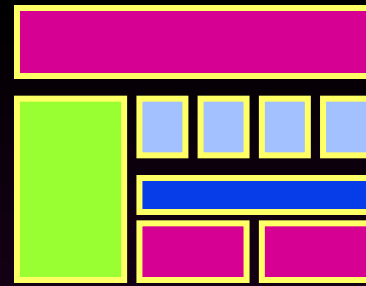
- Example:

```
StudentRecord Student1, Student2;
```



`Student1` and `Student2` are variables of `StudentRecord` type.

struct basics



- The members of a **struct** type variable are accessed with the dot (.) operator:

`<struct-variable>.<member_name>;`

- Example:

```
strcpy(Student1.Name, "Jahan Zeb");
Student1.Id = 12345;
strcpy(Student1.Dept, "COMP");
Student1.gender = 'M';
cout << "The student is ";
switch (Student1.gender) {
    case 'F': cout << "Ms. "; break;
    case 'M': cout << "Mr. "; break;
}
cout << Student1.Name << endl;
```

Student1

Name		
Id		Gender
Dept		

Jahan Zeb

12345

M

COMP

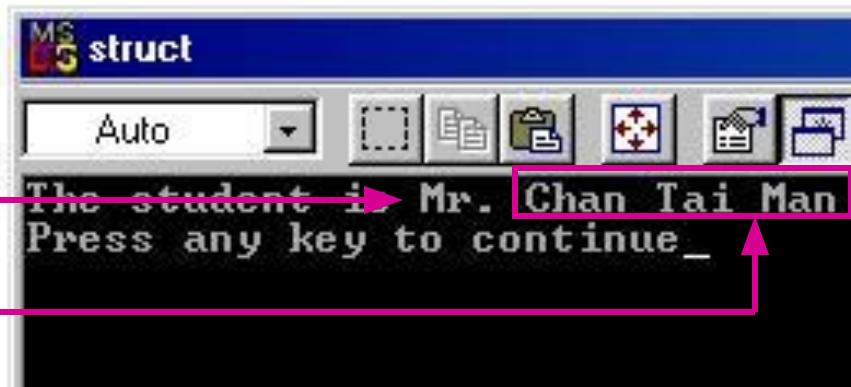
```
#include <string.h>

struct StudentRecord {
    char Name[22];
    int Id;
    char Dept[22];
    char gender;
};

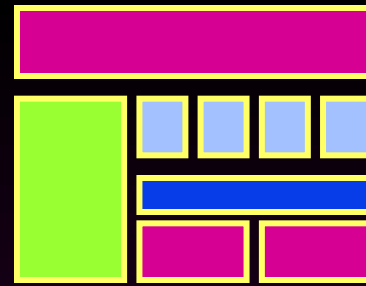
int main() {
    StudentRecord Student1;

    strcpy(Student1.Name, "Chan Tai Man");
    Student1.Id = 12345;
    strcpy(Student1.Dept, "COMP");
    Student1.gender = 'M';

    cout << "The student is ";
    switch (Student1.gender){
        case 'F': cout << "Ms. "; break;
        case 'M': cout << "Mr. "; break;
    }
    cout << Student1.Name << endl;
    return 0;
}
```



struct-to-struct assignment

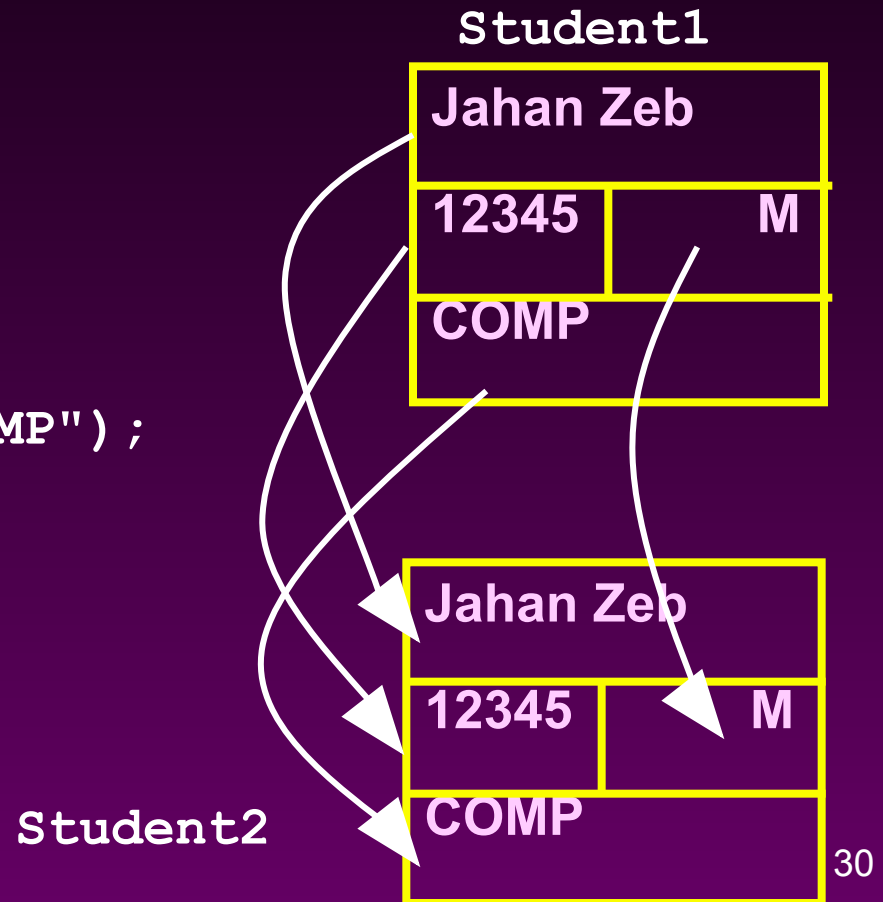


- The values contained in one struct type variable can be assigned to another variable of the same struct type.

- Example:

```
strcpy(Student1.Name,
        "Jahan Zeb");
Student1.Id = 12345;
strcpy(Student1.Dept, "COMP");
Student1.gender = 'M';
```

```
Student2 = Student1;
```



```

struct StudentRecord {
    char Name[22];
    int Id;
    char Dept[22];
    char gender;
};

int main() {
    StudentRecord Student1, Student2;

    strcpy(Student1.Name, "Chan Tai Man");
    Student1.Id = 12345;
    strcpy(Student1.Dept, "COMP");
    Student1.gender = 'M';

    Student2 = Student1;
    Student2.gender = 'F';

    cout << "The student is ";
    switch (Student2.gender){
        case 'F': cout << "Ms. "; break;
        case 'M': cout << "Mr. "; break;
    }
    cout << Student2.Name << endl;
    return 0;
}

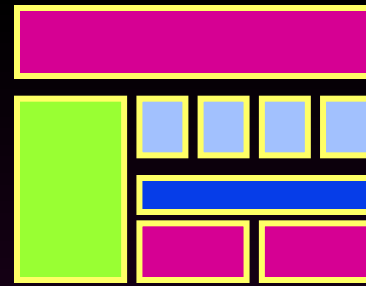
```

```

struct
Auto
The student is Ms. Chan Tai Man
Press any key to continue_

```

Nested structures

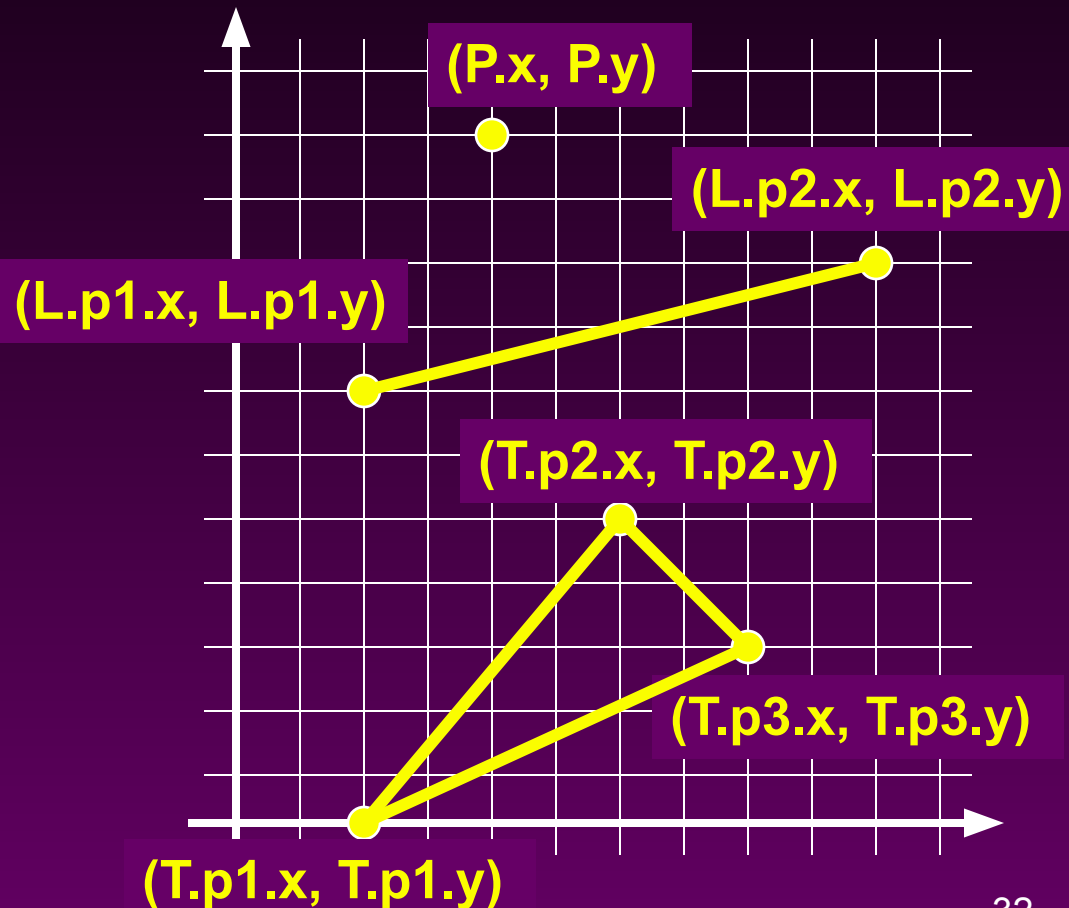


- We can nest structures inside structures.
- Examples:

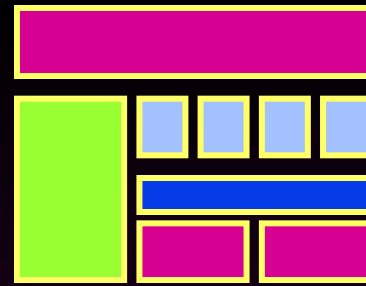
```
struct point{  
    double x, y;  
};  
point P;
```

```
struct line{  
    point p1, p2;  
};  
line L;
```

```
struct triangle{  
    point p1, p2, p3;  
};  
triangle T;
```

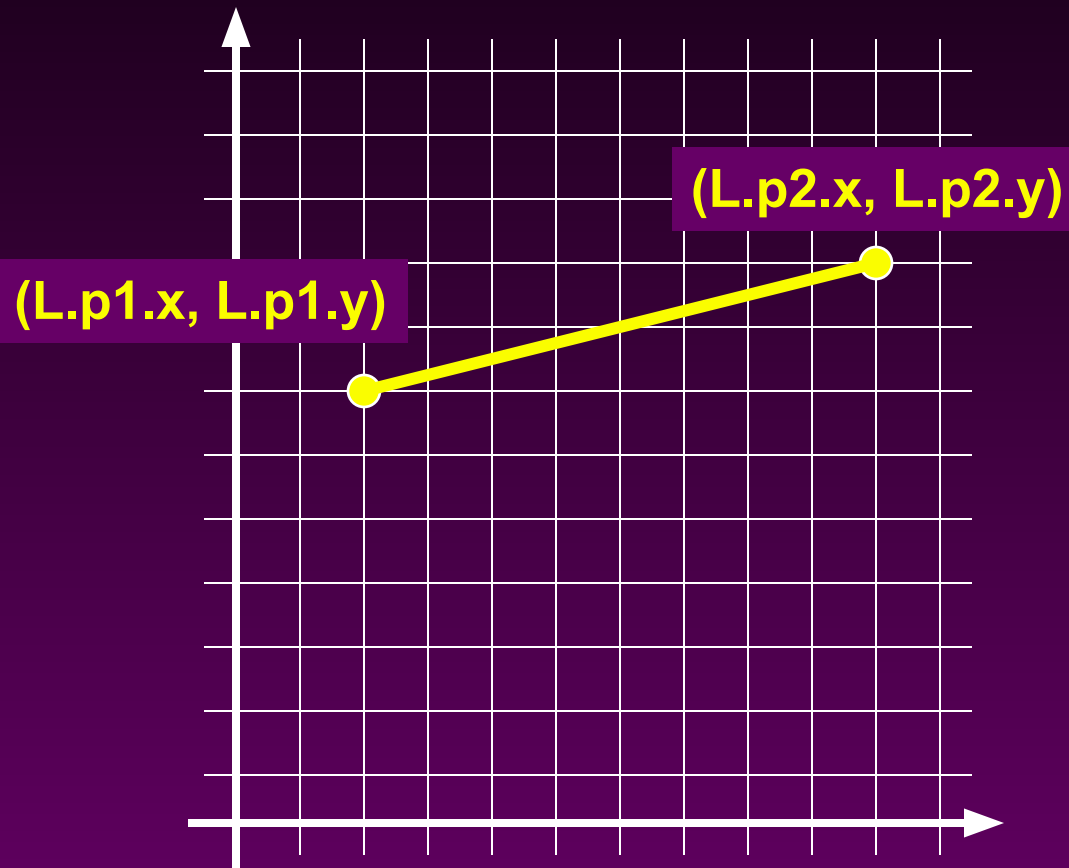
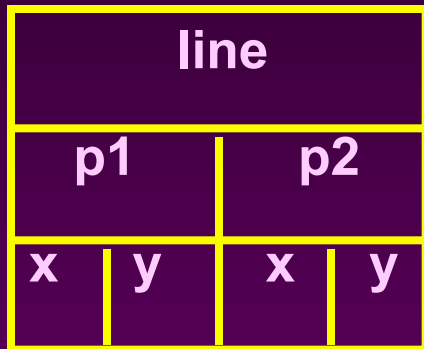


Nested structures

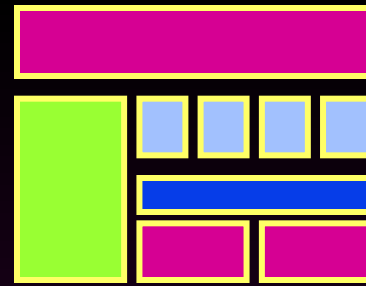


- We can nest structures inside structures.

```
• struct line{  
    point p1, p2;  
};  
line L;
```

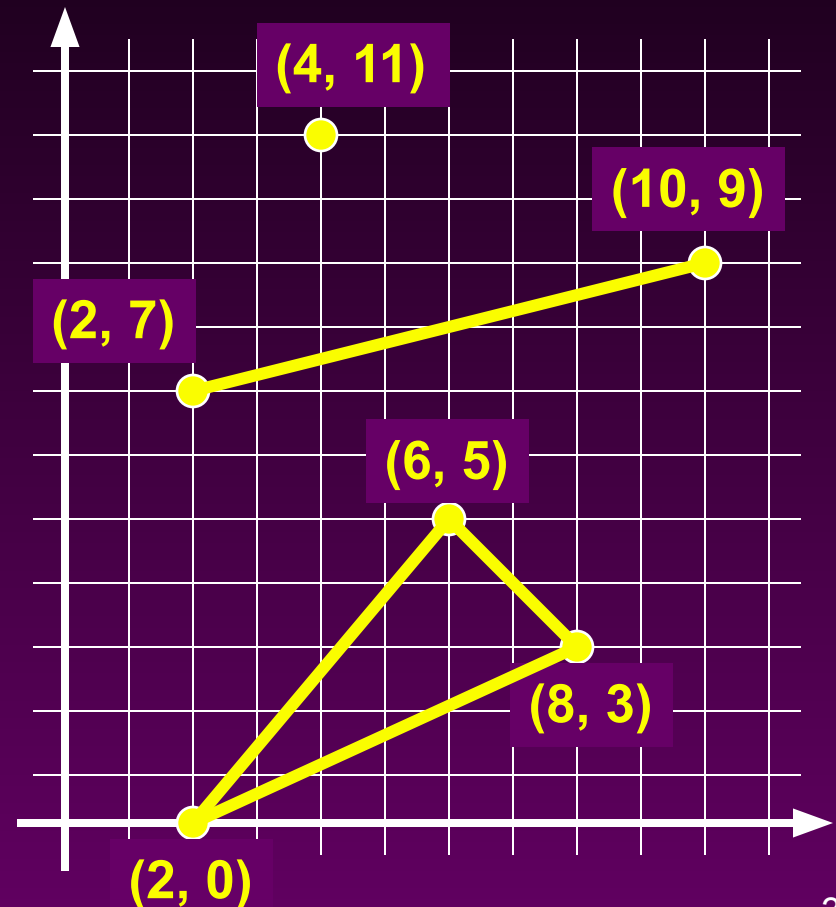


Nested structures

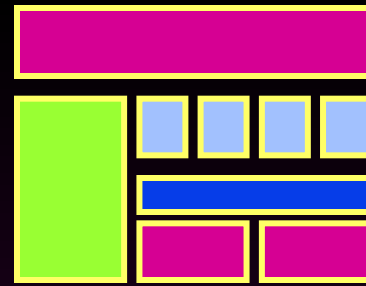


- Assign values to the variables `P`, `L`, and `T` using the picture:

```
point P;  
line L;  
triangle T;
```



Nested structures



```
point P;  
line L;  
triangle T;  
P.x = 4;  
P.y = 11;  
L.p1.x = 2;  
L.p1.y = 7;  
L.p2.x = 10;  
L.p2.y = 9;  
T.p1.x = 2;  
T.p1.y = 0;  
T.p2.x = 6;  
T.p2.y = 5;  
T.p3.x = 8;  
T.p3.y = 3;
```

